

Daily Workspace Automation

Project Roadmap & Requirements Document

Phase 1: Project Setup & Prerequisites

To start building this automation in your leisure time, begin by configuring your local development environment and gathering the necessary API credentials.

- **IDE:** Open VS Code and initialize a new project folder (e.g., workspace-automation).
- **Python Environment:** Create a virtual environment to keep dependencies isolated (`python -m venv venv`).
- **Google Cloud Console:** You will need a Service Account to interact with Google Sheets.
 - Go to the Google Cloud Console.
 - Enable the **Google Sheets API** and **Google Drive API**.
 - Create credentials for a **Service Account** and download the JSON key file. Place this securely in your project folder.
 - Share your specific attendance Google Sheet with the Service Account email address as an Editor.

Required Python Libraries:

Install these via pip: `pip install gspread oauth2client`

Standard libraries used: `os`, `shutil`, `subprocess`, `datetime`

Phase 2: System Cleanup (Temp & Prefetch)

The first programmatic task is clearing out temporary system files. This requires interacting with the OS file system.

Implementation Details:

- Use the built-in `os` and `shutil` modules.
- Define the target directories. On Windows 10, these are typically:
 - `C:\Windows\Temp`

- `C:\Users\<YourUsername>\AppData\Local\Temp` (Accessible via `os.environ.get('TEMP')`)
- `C:\Windows\Prefetch`
- Iterate through the contents of each directory using `os.listdir()` and remove files with `os.remove()` or directories with `shutil.rmtree()`.

Important: Modifying `C:\Windows\Temp` and `C:\Windows\Prefetch` requires administrative privileges. You will need to run your final script or terminal as an Administrator. Implement error handling (`try...except PermissionError`) to skip files currently in use by the OS.

Phase 3: Launching Applications & Browser Profiles

Next, automate the launching of your professional toolkit: Slack, Time Doctor, and a specific Chrome profile.

Implementation Details:

- Use the `subprocess.Popen()` method to start executable files without halting your Python script's execution.
- **Slack & Time Doctor:** Locate the exact `.exe` paths on your system.

```
subprocess.Popen([r"C:\Path\To\Slack.exe"])
```

- **Chrome Profiles:** To launch Chrome with your company's Gmail account rather than your personal one, you must specify the profile directory. Open `chrome://version/` in your work Chrome instance to find the "Profile Path" (e.g., "Profile 1" or "Default").

```
subprocess.Popen([r"C:\Program Files\Google\Chrome\Application\chrome.exe", '--profile-directory="Profile 1"'])
```

Phase 4: Google Sheets Attendance Logging

This phase handles the dynamic inputs for your check-in, break, and check-out times, alongside the reasons.

Implementation Details:

- **Authentication:** Use `oauth2client.service_account` to authenticate using your downloaded JSON key.
- **Connection:** Use `gspread.authorize()` to connect, then open the sheet by its title or URL.
- **Logic Flow:**
 - Use the `datetime` module to get the current timestamp formatted exactly as your company requires (e.g., `datetime.now().strftime("%Y-%m-%d %H:%M:%S")`).
 - Use Python's `input()` function to prompt yourself in the terminal for the current status ("check-in", "break-start", "break-end", "check-out") and the reason box.
 - Use `sheet.append_row([date, time, status, reason])` or target specific cells using `sheet.update()` based on how your attendance ledger is structured.

Phase 5: Shell Scripts & Automation OS Integration

To run this entire Python sequence cleanly, wrap it in a shell script. Here is how to configure it across different operating systems.

Windows (PowerShell)

Create a `start_work.ps1` file to execute the Python script. You can pin a shortcut to this PowerShell script to your taskbar.

```
# start_work.ps1 cd "C:\Path\To\Your\workspace-automation" # Request Admin rights if
not present if (-Not ([Security.Principal.WindowsPrincipal]
[Security.Principal.WindowsIdentity]::GetCurrent()).IsInRole([Security.Principal.Windo
wsBuiltInRole]::Administrator)) { Start-Process powershell -ArgumentList "-File
`"$PSCommandPath`" -Verb RunAs exit } .\venv\Scripts\activate python main.py
```

Linux / macOS (Bash Equivalent)

If you adapt this for a Unix-based system later, the paths and temp cleanup commands will change (e.g., clearing `/tmp` or `~/ .cache` instead of Prefetch).

```
#!/bin/bash # start_work.sh cd /path/to/your/workspace-automation source venv/bin/
activate # Sudo required for system-level cache clearing sudo python3 main.py
```

On Linux/macOS, you would grant execution permissions using `chmod +x start_work.sh` and execute it from your terminal.